

Execution Context & Scope Internals

JavaScript Advanced — Senior Developer Level

1. Execution Context কী?

JavaScript যখন কোনো code execute করে, তখন সেই execution-এর জন্য একটি **Execution Context** তৈরি হয়।

সহজভাবে:

Execution Context হলো **JavaScript**-এর এমন একটি **runtime environment** যেখানে একটি **piece of code** **execute** করার জন্য প্রয়োজনীয় তথ্য ও **bindings** রাখা/ব্যবস্থাপনা করা হয়।

একটি function call হলে তার execution-এর জন্য নতুন context তৈরি হয়।

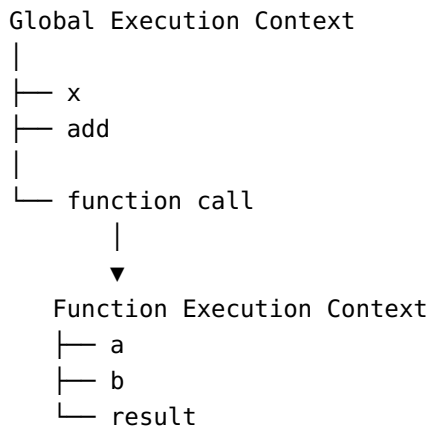
উদাহরণ:

```
const x = 10;

function add(a, b) {
  const result = a + b;
  return result;
}

add(5, 3);
```

Conceptually:



2. Execution Context-এর প্রধান ধরনের ধারণা

JavaScript execution model-এ বিভিন্ন ধরনের execution context থাকতে পারে।

প্রধানভাবে:

```
Execution Context
|
├── Global Execution Context
|
├── Function Execution Context
|
└── Eval Execution Context
```

Modern JavaScript-এ module code-এর জন্যও আলাদা execution semantics আছে।

তুমি developer হিসেবে সবচেয়ে বেশি কাজ করবে:

- Global context
- Function context
- Module context

নিয়ে।

3. Global Execution Context

যখন JavaScript program শুরু হয়, initial global execution context তৈরি হয়।

উদাহরণ:

```
const name = "Shariar";

function greet() {
  console.log("Hello");
}

console.log(name);
```

Conceptually:

Global Execution Context

- |
- ├─ name
- ├─ greet
- └─ global code

তারপর:

```
console.log(name)
```

execute হয়।

4. Function Execution Context

যখন function call হয়:

```
function add(a, b) {  
  const result = a + b;  
  return result;  
}  
  
add(10, 20);
```

function call-এর সময় নতুন execution context তৈরি হয়।

Global Context

- |
- | add(10, 20)



Function Context

- |
- ├─ a = 10
- ├─ b = 20
- └─ result = 30

Function execution শেষ হলে সেই context আর active থাকে না।

5. Execution Context-এর দুইটি গুরুত্বপূর্ণ Phase

একটি execution context-এর lifecycle বোঝার জন্য দুইটি conceptual phase খুব useful:

Creation Phase
↓
Execution Phase

এগুলো JavaScript-এর behavior বোঝার জন্য অত্যন্ত গুরুত্বপূর্ণ।

6. Creation Phase

Execution শুরু হওয়ার আগে environment প্রস্তুত করা হয়।

Conceptually engine/runtime:

- bindings তৈরি করে
- scope সম্পর্কিত information প্রস্তুত করে
- function declarations-এর bindings তৈরি করে
- lexical environment তৈরি করে
- `this` -related information নির্ধারণ করে
- প্রয়োজনীয় execution metadata প্রস্তুত করে

তারপর actual code execute হয়।

7. Execution Phase

তারপর statements execute হতে থাকে।

উদাহরণ:

```
function test() {  
  const x = 10;  
  const y = 20;  
  
  return x + y;  
}
```

Conceptually:

```
Creation
↓
Prepare function environment
↓
Execution
↓
x = 10
↓
y = 20
↓
return x + y
```

8. Hoisting-এর সঙ্গে Creation Phase-এর সম্পর্ক

এই code দেখো:

```
console.log(a);
var a = 10;
```

Output:

```
undefined
```

এটাকে বুঝতে conceptual model:

```
Creation Phase
a → initialized with undefined

Execution Phase
console.log(a)
↓
undefined

a = 10
```

তাই `var` declaration-এর behavior hoisting-এর সঙ্গে সম্পর্কিত।

9. Function Declaration কেন আগে থেকেই available?

```
greet();

function greet() {
  console.log("Hello");
}
```

এটি কাজ করে।

Conceptually:

```
Creation Phase
  ↓
greet binding prepared
  ↓
Execution Phase
  ↓
greet()
```

অন্যদিকে:

```
greet();

const greet = function () {
  console.log("Hello");
};
```

এখানে একইভাবে কাজ করবে না।

কারণ `const` binding-এর initialization execution-এর relevant point-এর আগে হয়নি।

10. Temporal Dead Zone

```
console.log(x);

let x = 10;
```

এখানে:

ReferenceError

কারণ `x` lexical environment-এ binding হিসেবে exist করলেও initialization-এর আগে access করা হচ্ছে।

এই period:

```
let x
  |
  | TDZ
  |
  ▼
x = 10
```

হলো **Temporal Dead Zone (TDZ)**।

Important:

TDZ মানে variable exist করে না—এটা নয়।

বরং:

binding আছে, কিন্তু initialized state-এ পৌঁছায়নি।

11. Execution Context-এর Internal Structure

ECMAScript specification অনুযায়ী execution context-কে বিভিন্ন internal components দিয়ে model করা হয়।

একটি simplified mental model:

```
Execution Context
|
├── LexicalEnvironment
|
├── VariableEnvironment
|
├── PrivateEnvironment
|
├── Function / Script / Module related state
|
└── Realm / execution-related metadata
```

তবে implementation-এ engine এই specification concepts-কে সরাসরি C++ object হিসেবে একইভাবে implement করতেই হবে— এমন নয়।

এটা খুব important distinction:

Specification model $\neq$ exact engine implementation

12. Lexical Environment

Lexical Environment মূলত identifier resolution-এর জন্য environment record + outer environment reference-এর conceptual structure।

Simplified:

```
Lexical Environment
|
├── Environment Record
|   ├── x
|   ├── y
|   └── function
└── Outer Environment
```

Example:

```
const x = 10;

function test() {
  const y = 20;

  console.log(x + y);
}
```

Function-এর lexical environment:

```
Function Environment
|
├── y = 20
└── outer → Global Environment
```



```
|  
└─ x = 10
```

13. Scope Chain

এখন আসল magic:

```
const globalValue = 10;  
  
function outer() {  
  const outerValue = 20;  
  
  function inner() {  
    const innerValue = 30;  
  
    console.log(globalValue);  
    console.log(outerValue);  
    console.log(innerValue);  
  }  
  
  inner();  
}  
  
outer();
```

`inner()` থেকে:

```
inner scope  
  ↓  
outer scope  
  ↓  
global scope
```

এটাই scope chain-এর conceptual model ।

14. Scope Resolution কীভাবে হয়?

ধরো:

```
const a = 10;

function outer() {
  const b = 20;

  function inner() {
    console.log(a);
    console.log(b);
  }

  inner();
}
```

`inner()` -এ:

```
console.log(a);
```

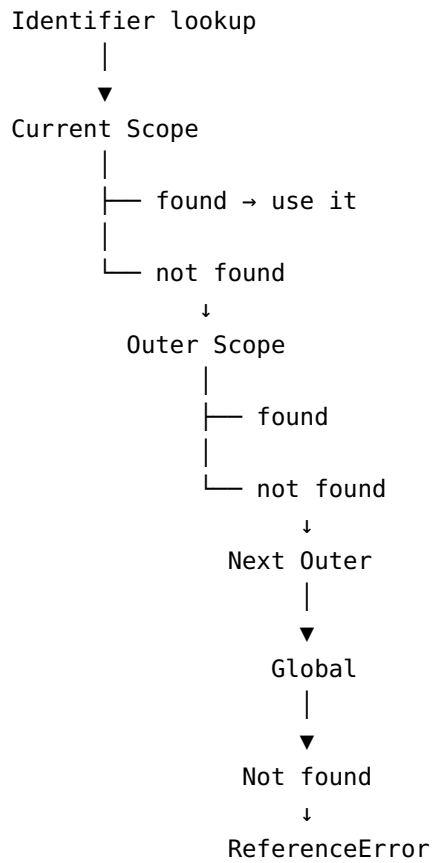
JavaScript প্রথমে current lexical environment-এ খুঁজবে:

```
inner
↓
a আছে?
↓
না
↓
outer
↓
a আছে?
↓
না
↓
global
↓
a আছে?
↓
হ্যাঁ
```

তারপর value পাওয়া যায়।

15. Local → Outer → Global

একটি useful mental model:



16. Shadowing

```
const name = "Global";

function test() {
  const name = "Local";

  console.log(name);
}

test();
```

Output:

Local

কারণ inner scope-এর `name` outer `name` -কে shadow করছে।

```
Global Scope
└─ name = "Global"

Function Scope
└─ name = "Local"
```

Lookup:

```
inner name
  ↓
found
  ↓
stop
```

Global `name` আর search করা হয় না।

17. Scope Chain ≠ Prototype Chain

দুটো গুলিয়ে ফেলবে না।

Scope Chain

Identifier lookup:

```
console.log(x);
```

Conceptually:

```
Local
  ↓
Outer
  ↓
Global
```

Prototype Chain

Property lookup:

```
object.name
```

Conceptually:

```
object
  ↓
prototype
  ↓
prototype's prototype
  ↓
null
```

দুটো completely different mechanism ।

18. Lexical Scope

JavaScript **lexically scoped** language ।

এর মানে:

কোন variable কোন scope-এ accessible হবে সেটা মূলত code-এর structure দ্বারা নির্ধারিত হয় ।

Example:

```
const x = "global";

function test() {
  console.log(x);
}
```

`test()` কোথা থেকে call হলো সেটা scope resolution-এর primary determinant নয় ।

বরং function যেখানে defined হয়েছে সেই lexical structure গুরুত্বপূর্ণ ।

19. Lexical Scope vs Dynamic Scope

Lexical Scope

```
const x = "global";

function a() {
  console.log(x);
}

function b() {
  const x = "local";
  a();
}

b();
```

Output:

global

কারণ `a()` -এর lexical environment `b()` নয়।

20. Dynamic Scope হলে

Dynamic scoping hypothetical হলে:

```
a()
↑
called from b()
↑
b-এর x
```

ব্যবহার হতে পারত।

কিন্তু JavaScript dynamic scope ব্যবহার করে না।

JavaScript:

```
where function is defined
  ↓
lexical environment
```

এর ওপর scope resolution নির্ভর করে।

21. Variable Environment বনাম Lexical Environment

এটি Advanced topic-এর সবচেয়ে confusing অংশগুলোর একটি।

Specification model-এ:

```
LexicalEnvironment
VariableEnvironment
```

দুইটি আলাদা component হিসেবে model করা হয়।

22. Variable Environment

Historically এবং specification model-এ `VariableEnvironment` মূলত `var` declarations এবং কিছু function-level binding-এর সঙ্গে সম্পর্কিত environment।

Example:

```
function test() {
  var x = 10;
}
```

Conceptually:

```
Variable Environment
└─ x
```

23. Lexical Environment

`let`, `const`, এবং block-scoped declarations-এর জন্য lexical environment অত্যন্ত গুরুত্বপূর্ণ।

```
function test() {
  let x = 10;

  if (true) {
    const y = 20;

    console.log(x);
    console.log(y);
  }
}
```

Conceptually:

```
Function Lexical Environment
└─ x

Block Lexical Environment
└─ y
```

Block শেষ হলে block environment আর directly accessible থাকে না।

24. `var` बनाम `let` Scope

```
function test() {
  if (true) {
    var a = 10;
    let b = 20;
  }

  console.log(a); // 10
  console.log(b); // ReferenceError
}
```

কারণ:

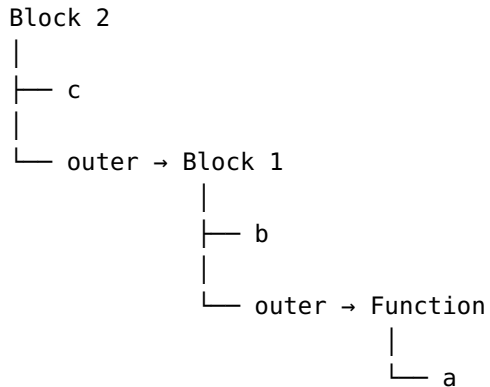
```
var
→ function scoped

let / const
→ block scoped
```

25. Nested Lexical Environments

```
function outer() {  
  const a = 10;  
  
  if (true) {  
    const b = 20;  
  
    if (true) {  
      const c = 30;  
  
      console.log(a);  
      console.log(b);  
      console.log(c);  
    }  
  }  
}
```

Conceptually:



এটাই nested lexical environment chain।

26. `catch` Block-ও আলাদা **Binding** তৈরি করতে পারে

```
try {  
  throw new Error("Oops");  
} catch (error) {
```

```
    console.log(error);  
  }
```

`error` binding catch-এর scope-এর সঙ্গে সম্পর্কিত।

Conceptually:

```
Catch Environment  
└─ error
```

27. Call Stack

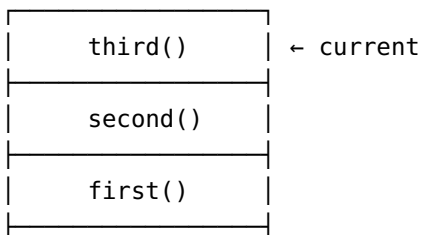
এখন Execution Context এবং Call Stack connect করি।

Call Stack হলো active execution contexts-এর runtime stack-এর simplified model।

Example:

```
function first() {  
  second();  
}  
  
function second() {  
  third();  
}  
  
function third() {  
  console.log("Hello");  
}  
  
first();
```

Stack:



global

28. Call Stack Frame কী?

প্রতিটি active function invocation-এর সঙ্গে একটি conceptual stack frame থাকে।

Frame-এর মধ্যে implementation অনুযায়ী থাকতে পারে:

- function-related state
- arguments
- local bindings-এর access
- return information
- execution metadata
- debugging information

Simplified:

```
Function Frame
|
├─ arguments
├─ local variables
├─ execution state
├─ return location
└─ scope-related information
```

কিন্তু আবার মনে রাখো:

এগুলো conceptual model; V8 exact physical representation implementation-dependent।

29. Function Call হলে কী হয়?

```
function add(a, b) {
  return a + b;
}
```

```
add(10, 20);
```

Conceptually:

Before call:

Call Stack

global

Call:

add(10, 20)

তারপর:

Call Stack

add()

global

add() return করলে:

Call Stack

global

30. Stack Overflow

যদি function endlessly নিজেকে call করে:

```
function recursion() {  
  recursion();  
}  
  
recursion();
```

তাহলে stack frame তৈরি হতে থাকবে:

```
recursion()  
recursion()  
recursion()  
recursion()  
recursion()  
...
```

শেষে runtime stack capacity exceed করলে সাধারণত:

```
RangeError: Maximum call stack size exceeded
```

এর মতো error পাওয়া যায়।

31. Recursion-এর Memory Model

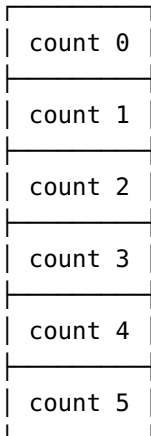
ধরো:

```
function count(n) {  
  if (n === 0) return;  
  
  count(n - 1);  
}  
  
count(5);
```

Conceptually:

```
count(5)  
↓  
count(4)  
↓  
count(3)  
↓  
count(2)  
↓  
count(1)  
↓  
count(0)
```

Stack:



32. Scope এবং Call Stack একই জিনিস নয়

এটা খুব important।

অনেকে ভাবে:

scope chain = call stack

না।

Example:

```
const x = 10;

function outer() {
  function inner() {
    console.log(x);
  }

  inner();
}

outer();
```

`inner()` -এর call stack:

```
inner
outer
global
```

Scope chain:

```
inner lexical environment
    ↓
outer lexical environment
    ↓
global lexical environment
```

এই example-এ দুটো দেখতে similar হলেও conceptually different ।

33. Closure এখানে কোথায় আসে?

```
function outer() {
  const secret = 42;

  return function inner() {
    return secret;
  };
}

const fn = outer();

console.log(fn());
```

`outer()` execution শেষ হয়ে গেলেও:

```
secret
```

এর value accessible ।

কারণ returned function-এর lexical environment `secret` -এর binding-এর সঙ্গে সম্পর্ক বজায় রাখে ।

Conceptually:

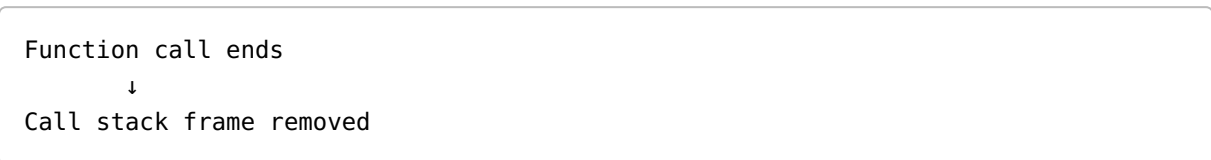
```
fn
|
```



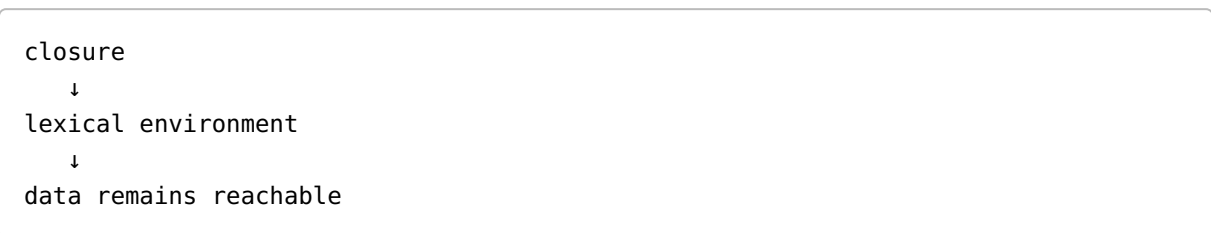
এটাই closure-এর core ।

34. Scope Lifetime বনাম Call Stack Lifetime

এখানে একটি subtle concept:



কিন্তু associated lexical environment-এর data যদি এখনও reachable হয়:



তাই closure-এর captured variable function return করার পরেও survive করতে পারে ।

35. Execution Context বনাম Scope

এগুলোও আলাদা ।

Execution Context

একটি code execution-এর runtime context ।

Scope

কোন identifier কোথা থেকে accessible সেটা নির্ধারণ করে।

সহজভাবে:

Execution Context

→ "এই code এখন execute করার environment"

Scope

→ "এই code কোন variables দেখতে পারবে"

দুটোর মধ্যে সম্পর্ক আছে, কিন্তু দুটো synonym নয়।

36. Tail Call কী?

ধরো:

```
function factorial(n, result = 1) {  
  if (n === 0) {  
    return result;  
  }  
  
  return factorial(n - 1, result * n);  
}
```

শেষ operation:

```
return factorial(...);
```

Recursive call-এর result নিয়ে পরে আর কোনো কাজ নেই।

এটাকে **tail call** বলা হয়।

37. Non-tail Recursive Call

```
function factorial(n) {  
  if (n === 0) return 1;  
  return factorial(n - 1) * n;  
}
```

```
    return n * factorial(n - 1);  
}
```

এখানে:

```
factorial(n - 1)  
    ↓  
return value  
    ↓  
multiply by n
```

Recursive call-এর পরে কাজ বাকি:

```
n * ...
```

তাই এটি tail position-এ নেই।

38. Tail Call Optimization (TCO)

TCO-এর idea:

যদি recursive/function call tail position-এ থাকে, তাহলে নতুন stack frame না রেখে existing frame reuse করা যেতে পারে।

Conceptually normal recursion:

```
frame 5  
frame 4  
frame 3  
frame 2  
frame 1
```

TCO থাকলে:

```
frame  
↓  
reuse  
↓  
reuse
```

↓
reuse

ফলে stack growth কমেতে পারে।

39. TCO-এর Benefit

Recursive algorithm:

```
function count(n) {  
  if (n === 0) return;  
  
  return count(n - 1);  
}
```

Without TCO:

```
n=100000  
↓  
huge number of stack frames  
↓  
stack overflow
```

With proper TCO:

```
single/reused frame conceptually  
↓  
continue  
↓  
continue
```

Stack space প্রায় constant রাখা সম্ভব।

40. Proper Tail Calls বনাম সাধারণ Optimization

এখানে একটা গুরুত্বপূর্ণ distinction:

Tail call এবং **Tail Call Optimization** এক জিনিস নয়।

Tail Call

→ code-এর property

TCO

→ runtime/compiler-এর optimization

অর্থাৎ code tail position-এ call করতে পারে, কিন্তু engine সেটিকে optimize করবেই—এমন নয়।

41. JavaScript-এ TCO-এর বাস্তব অবস্থা

ECMAScript specification proper tail calls-এর semantics define করেছে এবং strict-mode code-এর ক্ষেত্রে tail-position calls নিয়ে rules আছে।

কিন্তু major JavaScript engines-এর বাস্তব support historically inconsistent।

বিশেষ করে mainstream V8-based environments-এ সাধারণ developer code-এর জন্য TCO-কে dependable stack-saving optimization হিসেবে ধরে নেওয়া উচিত নয়।

তাই:

JavaScript recursion লিখে **stack overflow** এড়ানোর জন্য **TCO**-এর ওপর **rely** করা উচিত নয়।

Large recursion-এর ক্ষেত্রে সাধারণত:

- iterative approach
- explicit stack
- worker
- algorithm redesign

বিবেচনা করা ভালো।

42. Tail Recursion বনাম Iteration

Tail-recursive:

```
function sum(n, total = 0) {  
  if (n === 0) return total;  
  
  return sum(n - 1, total + n);  
}
```

Iterative:

```
function sum(n) {  
  let total = 0;  
  
  while (n > 0) {  
    total += n;  
    n--;  
  }  
  
  return total;  
}
```

Production JavaScript-এ দ্বিতীয় approach অনেক ক্ষেত্রে safer, কারণ stack growth-এর ওপর নির্ভর করতে হয় না।

43. Creation Phase + Scope + Stack — একসাথে

এখন পুরো বিষয় connect করি।

```
const x = 10;  
  
function outer(a) {  
  const y = 20;  
  
  function inner(b) {  
    return x + y + a + b;  
  }  
  
  return inner;  
}  
  
const fn = outer(30);  
  
fn(40);
```

Step 1 — Global context

```
Global Context  
|  
├─ x = 10
```

```
|— outer
|— fn
```

Step 2 — `outer(30)`

নতুন function execution context:

```
outer Context
|
|— a = 30
|— y = 20
|— inner
```

Stack:

```
outer
global
```

Step 3 — `outer()` returns

`outer()` execution শেষ।

Call stack:

```
global
```

কিন্তু `inner` function তার lexical environment-এর সঙ্গে relationship ধরে রাখে।

```
fn
↓
inner
↓
outer lexical environment
|— a = 30
|— y = 20
```

Step 4 — `fn(40)`

নতুন context:

```
inner Context
└─ b = 40
```

Scope lookup:

```
b → inner
a → outer
y → outer
x → global
```

তাই:

```
10 + 20 + 30 + 40
= 100
```

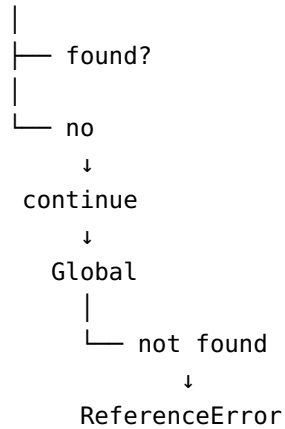
44. Scope Resolution-এর Complete Mental Model

যখন JavaScript দেখে:

```
console.log(value);
```

conceptually:

```
Identifier: value
  |
  ▼
Current Lexical Environment
  |
  ├── found?
  │   └─ yes → use binding
  └─ no
      ↓
Outer Environment
```



45. `this` Scope Chain-এর অংশ নয়

আরেকটি common confusion:

```
console.log(this);
```

`this` সাধারণ identifier lookup-এর মতো lexical scope chain দিয়ে resolve হয় না।

`this` value function invocation এবং execution context-এর rules-এর ওপর নির্ভর করে।

Arrow function-এর ক্ষেত্রে:

```
const obj = {
  value: 10,

  method() {
    const arrow = () => {
      console.log(this.value);
    };

    arrow();
  }
};
```

Arrow function নিজের `this` তৈরি করে না; surrounding context থেকে `this` নেয়।

তাই:

Scope chain এবং `this` binding আলাদা concept।

46. Scope Chain-এর Performance Consideration

Modern JavaScript engines scope access optimize করতে পারে।

তবে extremely dynamic code optimization কঠিন করতে পারে।

যেমন:

```
with (obj) {  
  console.log(value);  
}
```

`with` lexical lookup predictability নষ্ট করে।

এ কারণেই strict mode-এ `with` allowed নয়।

Similarly:

```
eval(...)
```

dynamic code injection-এর কারণে scope analysis এবং optimization complicate করতে পারে।

Senior-level rule:

Avoid `with` and avoid unnecessary dynamic `eval()`.

47. Closures এবং Memory

Closure নিজে memory leak নয়।

```
function createCounter() {  
  let count = 0;  
  
  return () => ++count;  
}
```

এখানে `count` survive করছে কারণ function-এর দরকার আছে।

Leak তখন হতে পারে যখন unnecessary reference দীর্ঘসময় ধরে data ধরে রাখে।

```
Long-lived object
  ↓
closure
  ↓
huge object
  ↓
cannot collect
```

তাই scope lifetime বোঝা memory management-এর সঙ্গেও directly connected ।

48. Execution Context — Practical Mental Model

একটি function call-এর সময় ভাবতে পারো:

```
Function Call
  ↓
Create execution environment
  ↓
Create/prepare bindings
  ↓
Resolve outer lexical environment
  ↓
Set invocation-related state
  ↓
Execute statements
  ↓
Return
  ↓
Context no longer active
```

কিন্তু closure থাকলে associated lexical state-এর কিছু অংশ এখনও reachable থাকতে পারে ।

49. Common Misconceptions

❌ "Every variable is stored on the stack."

না।

JavaScript engine-এর actual memory representation implementation-dependent ।

Specification-level bindings আর physical stack/heap location এক জিনিস নয় ।

❌ "Scope is the call stack."

না।

Scope lexical structure-এর সঙ্গে সম্পর্কিত।

Call stack active execution-এর সঙ্গে সম্পর্কিত।

❌ "Function return করলে সব local variables instantly disappear."

সাধারণ case-এ active frame শেষ হয়, কিন্তু closure বা অন্য reachable reference থাকলে associated data survive করতে পারে।

❌ "let/const creation phase-এ exist করে না."

Binding থাকতে পারে, কিন্তু initialization-এর আগে access করলে TDZ-এর কারণে error হয়।

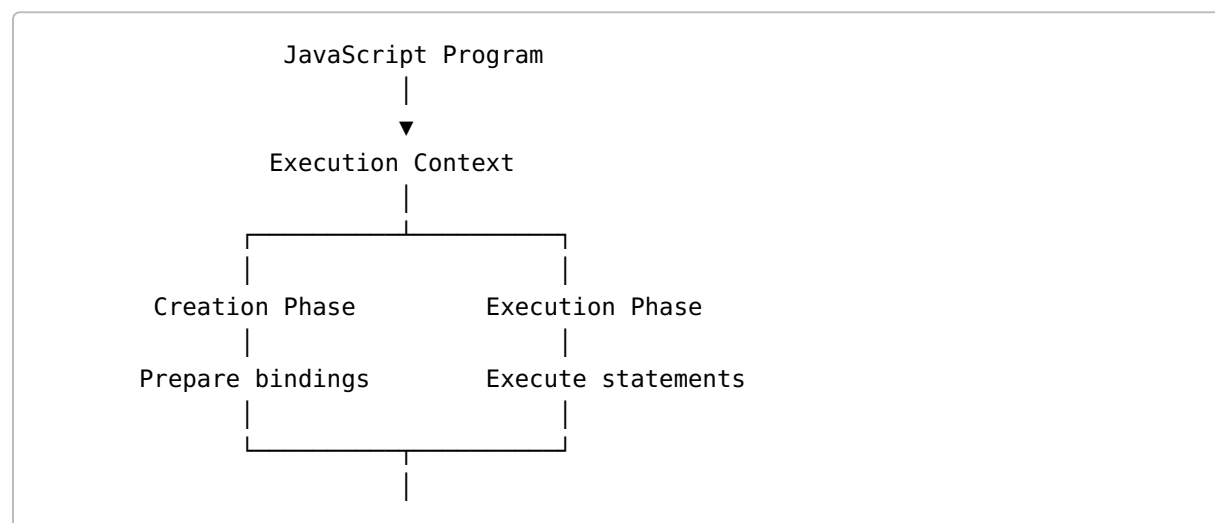
❌ "Tail recursion automatically prevents stack overflow."

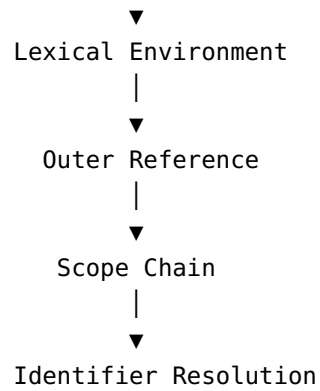
না।

Tail-call optimization runtime support-এর ওপর নির্ভর করে।

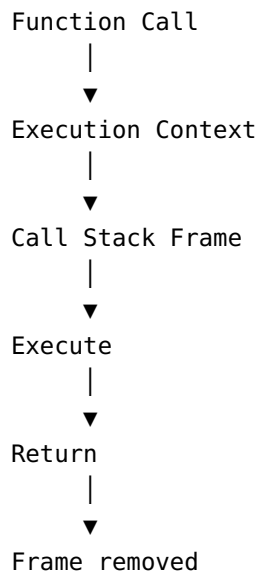
50. Senior Developer Mental Model

এই group-এর সবচেয়ে useful mental model:

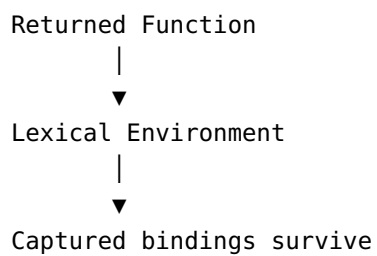




আর function call-এর জন্য:



Closure থাকলে:



51. Quick Revision

Concept	Meaning
Execution Context	code execute করার runtime environment
Creation Phase	execution-এর আগে environment/bindings প্রস্তুত করার conceptual phase
Execution Phase	actual statements execute হয়
Lexical Environment	identifier bindings + outer environment সম্পর্কিত structure
Variable Environment	specification model-এর variable/function declaration-related environment component
Scope Chain	current → outer → global lookup path
Lexical Scope	code structure দ্বারা scope নির্ধারিত
Shadowing	inner binding outer binding-কে hide করে
Call Stack	active execution-এর stack
Stack Frame	active function invocation-এর conceptual frame
Closure	function + তার lexical environment-এর relationship
Tail Call	return করার আগে function call-ই শেষ operation
TCO	tail call-এর জন্য stack reuse করার optimization

52. Interview Questions

Fundamental

1. What is an execution context?
2. What happens during the creation phase?
3. What happens during the execution phase?
4. What is lexical scope?
5. What is a scope chain?

Advanced

1. Lexical Environment কী?
2. Variable Environment বনাম Lexical Environment?
3. `var`, `let`, `const` scope behavior কীভাবে আলাদা?
4. TDZ কী এবং কেন হয়?
5. Scope chain এবং prototype chain-এর পার্থক্য কী?
6. Execution context এবং call stack-এর relationship কী?

7. Stack frame-এর মধ্যে কী ধরনের information থাকতে পারে?
8. Closure function return করার পরেও variable কীভাবে access করে?
9. Scope chain এবং call stack কি একই?
10. Tail call কী?
11. Tail Call Optimization কী?
12. JavaScript-এ TCO-এর ওপর কেন rely করা উচিত নয়?
13. `async/await` কি call stack-এর normal synchronous frame-এর মতো কাজ করে?
14. `eval()` optimization এবং scope analysis-কে কেন complicate করতে পারে?
15. JavaScript engine কীভাবে lexical scope access optimize করতে পারে?

53. Final Takeaways

এই পুরো chapter থেকে মূল ৭টা জিনিস মনে রাখো:

1. Execution Context
 - code execution-এর runtime environment
2. Creation Phase
 - execution-এর আগে bindings/environment প্রস্তুত
3. Lexical Environment
 - bindings + outer environment relationship
4. Scope Chain
 - current → outer → global identifier lookup
5. Call Stack
 - বর্তমানে active execution-এর stack
6. Closure
 - function তার lexical environment-এর সঙ্গে relationship ধরে রাখতে পারে
7. TCO
 - tail call-এর stack growth কমানোর optimization concept

সবশেষে সবচেয়ে important distinction:

Scope
≠
Call Stack

Execution Context
≠

Scope

Lexical Environment

≠

Physical JavaScript Heap/Stack location

Tail Call

≠

Tail Call Optimization

Senior JavaScript developer হিসেবে এগুলো আলাদা করে চিনতে পারাটাই আসল।